

Theory Assignment – 4

1. What is a closure in JavaScript? Explain with a suitable example.

A **closure** is a function that remembers and can access variables from its outer (lexical) scope even after the outer function has finished executing.

Example:

```
function outer() {  
    let count = 0;  
  
    function inner() {  
        count++;  
        console.log(count);  
    }  
  
    return inner;  
}  
  
const counter = outer();  
  
counter(); // 1  
counter(); // 2
```

Here, `inner()` forms a closure over the `count` variable. Even after `outer()` has finished execution, `inner()` can still access and modify `count`.

2. What is a Promise in JavaScript? Explain its different states.

A **Promise** is an object that represents the eventual completion or failure of an asynchronous operation.

A Promise has three states:

1. **Pending** – The operation has not completed yet.
2. **Fulfilled** – The operation completed successfully.
3. **Rejected** – The operation failed.

Example:

```
const promise = new Promise((resolve, reject) => {
  let success = true;

  if (success) {
    resolve("Operation successful");
  } else {
    reject("Operation failed");
  }
});

promise
  .then(result => console.log(result))
  .catch(error => console.log(error));
```

3. What is async/await in JavaScript? How does it simplify working with Promises?

async/await is a syntax used to work with Promises in a simpler and more readable way.

- **async** makes a function return a Promise.
- **await** pauses the execution of an **async** function until a Promise is settled.

Example:

```
async function getData() {
  try {
    const response = await fetch("https://example.com/data");
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.log(error);
  }
}
```

It simplifies Promise handling because asynchronous code can be written in a style that looks similar to synchronous code, avoiding long chains of `.then()`.

4. What is the Global Execution Context in JavaScript?

The **Global Execution Context (GEC)** is the default execution context created by JavaScript when a program starts.

It is responsible for:

- Creating the global object.
- Creating the global `this` value.
- Setting up memory for global variables and functions.
- Executing global-level JavaScript code.

The Global Execution Context is created before the execution of the JavaScript program begins.

5. What is currying in JavaScript? Explain with an example.

Currying is a technique of converting a function that takes multiple arguments into a sequence of functions, where each function takes one argument.

Example:

```
function add(a) {  
  return function(b) {  
    return function(c) {  
      return a + b + c;  
    };  
  };  
}
```

```
const result = add(10)(20)(30);
```

```
console.log(result);
```

Output:

60

Here, instead of passing three arguments at once, the arguments are passed one at a time.

6. What is jQuery, and what is its significance in web development?

jQuery is a fast and lightweight JavaScript library designed to simplify tasks such as DOM manipulation, event handling, animations, and AJAX operations.

Example:

```
$("#btn").click(function() {  
    $("#message").text("Hello World");  
});
```

Significance of jQuery:

- Simplifies DOM manipulation.
- Makes event handling easier.
- Provides simple AJAX methods.
- Provides animation effects.
- Reduces the amount of JavaScript code required.
- Provides cross-browser support.

7. What is a component in React? Explain its types.

A **component** in React is a reusable and independent piece of UI that can accept data and return a React element describing what should appear on the screen.

The main types are:

1. Functional Component

A JavaScript function that returns JSX.

```
function Welcome() {  
    return <h1>Hello World</h1>;  
}
```

2. Class Component

A JavaScript class that extends `React.Component`.

```
class Welcome extends React.Component {  
  render() {  
    return <h1>Hello World</h1>;  
  }  
}
```

Functional components are commonly used in modern React applications.

8. What is the Virtual DOM? How does it improve application performance?

The **Virtual DOM** is a lightweight JavaScript representation of the actual DOM maintained by React.

When the state of a React application changes:

1. React creates an updated Virtual DOM.
2. It compares the new Virtual DOM with the previous one.
3. It identifies the differences.
4. It updates only the necessary parts of the actual DOM.

This reduces unnecessary DOM operations and can improve application performance, especially when many UI elements are involved.

9. What is JSX? Why can browsers not understand it directly?

JSX (JavaScript XML) is a syntax extension for JavaScript commonly used with React. It allows HTML-like syntax to be written inside JavaScript.

Example:

```
const element = <h1>Hello World</h1>;
```

Browsers cannot understand JSX directly because JSX is **not standard JavaScript syntax**. It must first be transformed into regular JavaScript by tools such as **Babel** or a framework's build system.

For example:

```
const element = <h1>Hello World</h1>;
```

is transformed into JavaScript that creates the corresponding React element.

10. What are props in React? Are they mutable or immutable?

Props (properties) are data passed from a parent component to a child component in React.

Example:

```
function Welcome(props) {  
  return <h1>Hello, {props.name}</h1>;  
}  
  
function App() {  
  return <Welcome name="Rahul" />;  
}
```

Here, name is a prop passed from App to Welcome.

Props are immutable (read-only) from the child component's perspective. A child component should not directly modify its props. If the data needs to change, the parent component should update the data and pass the new value as props.